

Relatório Técnico do Experimento RAG-InsurMinds

Aviso Importante

- Os arquivos oficiais deste projeto estão em: <https://github.com/marcelobazevedo/RAG-InsurMinds>
- Para executar a solução descrita neste relatório, é necessário baixar/clonar esse repositório.
- A execução da aplicação depende de ambiente Docker (Docker Engine + Docker Compose).

1. Resumo Executivo

Este relatório descreve o experimento de construção e consolidação de uma aplicação RAG (Retrieval-Augmented Generation) para o domínio de seguro residencial, implementada no repositório **RAG-InsurMinds**.

A solução final entrega um fluxo operacional completo:

1. upload de documentos PDF;
2. ingestão automática e indexação vetorial em PostgreSQL + pgvector;
3. recuperação híbrida de contexto (**dense, sparse, hybrid**);
4. geração de resposta em streaming com citação de fontes;
5. operação de limpeza completa (arquivos + base vetorial) via interface.

Para operação, os PDFs da pasta **documentos_para_upload/** devem ser enviados pela interface; após envio, ficam na pasta operacional **documentos/**.

A arquitetura foi containerizada com Docker Compose, mantendo também execução local com **uv**.

2. Objetivo do Experimento

O objetivo técnico foi validar um pipeline RAG de ponta a ponta que permitisse:

- transformar PDFs de negócio em base consultável;
 - responder perguntas com grounding em evidências recuperadas;
 - suportar operação simples por interface web;
 - manter flexibilidade de provedor de modelo (local via Ollama ou OpenAI);
 - garantir rastreabilidade da resposta por fonte.
-

2.1 Vídeo de Demonstração

Para registro da operação da aplicação, o experimento também possui um vídeo de demonstração:

- <https://drive.google.com/file/d/1GF7AtyAyOZ2fKrEI6nbRF6IB2XikERXc/view?usp=sharing>
-

3. Escopo e Fontes de Verdade

A descrição deste relatório foi construída a partir dos artefatos do próprio projeto:

- [README.md](#)
 - [TECNICO.md](#)
 - [app.py](#)
 - módulos em [rag/](#) (ingestão, recuperação, geração, guardrails)
 - infraestrutura Docker ([Dockerfile](#), [docker-compose.yaml](#))
 - diagrama técnico: [desenho_tecnico_rag_insurminds.png](#)
-

4. Arquitetura da Solução

A arquitetura está representada no diagrama [desenho_tecnico_rag_insurminds.png](#), com quatro camadas principais:

1. Camada de Interface

- Streamlit ([app.py](#)) com chat, upload e exibição de fontes.

2. Camada de Ingestão

- leitura de PDF;
- chunking com metadados;
- geração de embeddings;
- sanitização de texto/metadados;
- persistência no banco.

3. Camada de Dados

- PostgreSQL com extensão pgvector;
- tabela [dados](#) para texto + metadados + vetor.

4. Camada RAG

- recuperação [dense](#) (vetorial), [sparse](#) (BM25) e [hybrid](#) (RRF);
- guardrail de suficiência de evidência;
- geração de resposta por LLM com streaming.

4.1 Infraestrutura de execução

No [docker-compose](#) há dois serviços:

- [rag_insurminds_postgres](#) (pgvector);
- [rag_insurminds_streamlit](#) (aplicação).

Também foi configurado bind mount para documentos:

- [./documentos:/app/documentos](#)
- [./documentos:/app/static](#)

Isso garante que:

- arquivos enviados pela UI persistam no host;
- links de fontes possam abrir os PDFs via rota estática do Streamlit ([/app/static/...](#)).

5. Pipeline Implementada

5.1 Fluxo A - Upload e Ingestão

Fluxo efetivo implementado:

1. usuário envia PDF na sidebar;
2. validações de entrada:
 - extensão `.pdf`;
 - tamanho máximo 50 MB;
 - bloqueio de nome duplicado;
3. arquivo (selecionado da pasta `documentos_para_upload/`) salvo em `documentos/`;
4. ingestão automática do arquivo recém enviado;
5. criação/garantia de tabela `dados`;
6. chunking + metadados + embeddings;
7. gravação no PostgreSQL com deduplicação por `chunk_id`.

5.1.1 Detalhes de ingestão

Em `extract_text.py`:

- extração textual com `pypdf`;
- divisão em chunks com `CHUNK_SIZE` e `CHUNK_OVERLAP`;
- tentativa de seccionamento por marcadores do domínio (`coberturas`, `exclusoes`, `franquia`, `sinistro`, `assistencia`);
- inferência de metadados como `doc_id`, `titulo`, `numero_documento`, `pdf_name`, `chunk_type` e `chunk_index`.

Em `pgvector_store.py`:

- tabela `dados` com coluna `embedding VECTOR(PGVECTOR_DIM)`;
- inserção com `ON CONFLICT (chunk_id) DO NOTHING`;
- sanitização de `\x00` (NUL) para evitar falhas de escrita no banco.

5.2 Fluxo B - Pergunta e Resposta

Fluxo efetivo implementado:

1. usuário envia pergunta;
2. `run_streaming_rag()` inicia execução do grafo;
3. nó `retrieve` retorna documentos recuperados;
4. nó `generate` gera resposta em streaming;
5. UI exibe tokens em tempo real e lista de fontes.

5.2.1 Recuperação

Em `retriever.py`:

- **dense_search**: similaridade vetorial por produto interno;
- **bm25_search**: ranking lexical com BM25;
- **hybrid_search**: fusão via Reciprocal Rank Fusion.

Em [retrieval_node.py](#):

- expansão opcional de query com LLM, especializada em termos de seguro residencial;
- recuperação para modos **dense**, **sparse**, **hybrid**;
- retorno de metadados de busca para painel de detalhes da UI.

5.2.2 Geração e Grounding

Em [augmented_node.py](#):

- prompt com política estrita de grounding;
- instrução explícita para citação em formato **[doc_id#chunk_id]**;
- fallback textual quando não houver base.

5.3 Fluxo C - Limpeza Operacional

A UI inclui botão “Remover todos os arquivos”, que executa:

1. remoção de PDFs em **documentos/**;
2. **TRUNCATE TABLE dados RESTART IDENTITY** no PostgreSQL.

Esse fluxo reduz inconsistência entre filesystem e índice vetorial.

6. Modelo de Dados

Tabela principal: **dados**

Campos:

- **id** (serial, PK)
- **chunk_id** (text, unique)
- **doc_id** (text)
- **titulo** (text)
- **fonte** (text)
- **data** (text)
- **tipo** (text)
- **text** (text)
- **metadata** (jsonb)
- **embedding** (**vector(PGVECTOR_DIM)**)

Observações:

- **chunk_id** funciona como chave lógica para deduplicação;
 - **metadata** concentra rastreabilidade de origem;
 - a dimensão do vetor deve ser compatível com **PGVECTOR_DIM**.
-

7. Estratégia de Modelos (Local e OpenAI)

Em `model_provider.py`, a escolha do backend é controlada por `MODELO_LOCAL`.

7.1 Modo local (`MODELO_LOCAL=true`)

- chat e embedding via Ollama;
- variáveis: `LLM_MODEL`, `EMBEDDING_MODEL`, `OLLAMA_URL`.

7.2 Modo OpenAI (`MODELO_LOCAL=false`)

- chat via `OPENAI_MODEL`;
- embedding via `OPENAI_EMBEDDING_MODEL`;
- autenticação por `OPENAI_API_KEY` em `.env`.

Validações de variáveis obrigatórias são feitas antes da inferência.

8. Reprodutibilidade e Operação

8.1 Execução em container

Comandos principais:

```
docker compose up -d --build
docker compose ps
docker compose logs -f streamlit
docker compose logs -f postgres
```

8.2 Execução local com uv

```
uv sync
uv run streamlit run app.py
```

8.3 Ingestão manual (opcional)

```
docker compose exec streamlit uv run --no-sync python -m
rag.ingest.extract_text
```

9. Decisões Técnicas Relevantes

1. Escolha por recuperação híbrida (`dense` + `sparse` + RRF)
- melhora robustez em perguntas semânticas e literais.

2. Ingestão automática no upload

- reduz atrito operacional para usuários não técnicos.

3. Sanitização de caracteres NUL

- corrige falha comum de PDFs malformados.

4. Limpeza unificada (arquivos + banco)

- mantém consistência entre conteúdo físico e índice vetorial.

5. Exibição de fontes por resposta

- melhora auditabilidade e confiança no resultado.
-

10. Limitações Observadas

1. Custo/latência de embeddings em tempo de upload

- em documentos grandes, ingestão pode demorar proporcionalmente ao número de chunks.

2. Chunking heurístico

- baseado em marcadores textuais; PDFs muito desestruturados podem reduzir qualidade da segmentação.

3. Guardrail desacoplado da resposta final no frontend

- o módulo de avaliação de suficiência existe no código, mas o ciclo de UX atual privilegia resposta + fontes sem painel específico de decisão do guardrail.

4. Dependência da qualidade do OCR/texto original

- documentos escaneados ou com extração ruim podem prejudicar recall e resposta.
-

11. Riscos e Mitigações

1. Erro de dimensão de embedding

- mitigado por validação explícita em `PgVectorStore.add_texts()`.

2. Erro de conexão com banco

- mitigado por documentação operacional e separação de portas host/container.

3. Link de fonte quebrado

- mitigado por mount de `documentos` também em `/app/static` e construção de link compatível com Streamlit.

4. Respostas sem evidência forte

- mitigado por política de grounding, citação obrigatória e heurísticas de suficiência.
-

12. Conclusão

O experimento atingiu o objetivo de disponibilizar um RAG funcional, operacional e rastreável para o domínio de seguro residencial, com:

- pipeline automatizada de ingestão;
- indexação vetorial em pgvector;
- recuperação híbrida;
- geração com grounding e citação;
- operação simplificada por interface e containerização.

A solução está pronta para uso interno e para evolução incremental (ex.: frontend React/API dedicada, observabilidade, métricas online e avaliação contínua com dataset curado do domínio).

13. Artefatos Principais

- Aplicação: [app.py](#)
- Ingestão: [extract_text.py](#), [pgvector_store.py](#)
- Recuperação: [retriever.py](#), [retrieval_node.py](#)
- Geração: [augmented_node.py](#)
- Model provider: [model_provider.py](#)
- Infra: [docker-compose.yaml](#), [Dockerfile](#)
- Documentação base: [README.md](#), [TECNICO.md](#)
- Figura de arquitetura: [desenho_tecnico_rag_insurminds.png](#)